Software Engineering Lab 3

Component Modelling & Architecture Selection

Name: Dhanush Gowda A S **SRN:** PES1UG25CS812

Section: G

Problem Statement #60: Podcast Guest Scheduling & Outline Builder

Architecture Selection

For the Podcast Guest Scheduling & Outline Builder, we chose Layered Architecture. The system allows podcast hosts to publish interview availability, guests to book suitable time slots, and guests to submit structured talking-point outlines. Hosts can review and edit these outlines before the interview, and the system generates timestamped run-of-show production notes from the approved outline.

1. Architectural Choice

Selected Architectural Style: Layered Architecture

We selected Layered Architecture because it divides the system into layers with distinct responsibilities.

- **Presentation Layer:** Provides the Podcast Guest Web UI and Show Host Dashboard. Guests view available slots, book interviews, and submit talking points. Hosts manage availability, review bookings, and edit outlines.
- **Business Logic Layer:** Contains the scheduling service, topic-outline manager, run-of-show generator, authentication/session management, and calendar invitation service. It validates bookings, prevents double booking, manages outline review, and generates timestamped production notes.
- **Data Layer:** Stores host availability, interview bookings, guest-submitted and host-edited outlines, episode production notes, and related status information.

The presentation layer communicates with application services through defined interfaces. Application services access persistent information through the data layer, while external calendar and email services are integrated through dedicated components.

2. Reasons for Choosing Layered Architecture

Reason 1 – Clear Separation of Different Operations

The application handles several activities: publishing availability, checking open time slots, confirming bookings, sending calendar invitations, collecting talking points, reviewing outlines, and generating production notes. Keeping these operations in separate components makes the system easier to understand, test, and maintain. Each component can focus on its own responsibility and communicate through defined interfaces.

Reason 2 – Easier Maintenance and Future Expansion

The system may later support multiple hosts, additional calendar providers, reminder notifications, reusable outline templates, or different production-note formats. With Layered Architecture, changes can usually be made in the relevant component without redesigning the entire application. For example, a new calendar provider can be integrated through the calendar integration component while the booking workflow remains mostly unchanged.

3. Security Advantage

Layered Architecture improves security by ensuring that user requests do not directly change important booking or outline records. Requests pass through application logic, where identity, permissions, and business rules can be checked before data is updated.

- Authentication and session management: verifies users and maintains authorized sessions.
- Booking validation: checks that a time slot is still available before confirming it, helping prevent double booking.
- Role-based access: limits host-only actions, such as editing submitted outlines and managing availability, to authorized hosts.
- Data protection and auditability: keeps booking and outline data in the data layer and can record important booking or outline changes.

4. Performance Benefit

Layered Architecture allows frequent operations to be handled by dedicated services. The Interview Scheduling Service can check availability and confirm bookings without involving the run-of-show generation logic. The Topic Outline Manager can validate and retrieve outlines independently, while the Run-of-Show Generator can focus on arranging approved topics and adding timestamps. Separating these responsibilities makes individual components easier to optimize and helps keep booking and outline-submission workflows responsive.

5. Component Diagram

The component diagram below represents the proposed Layered Architecture for Problem Statement #60. The Podcast Guest and Show Host interact with the User Interface subsystem. Requests pass to application components for authentication, scheduling, outline management, run-of-show generation, and calendar invitations. External services support calendar and email delivery. The Data Management subsystem stores availability and booking data, topic outlines, and production notes, while the Notification Service coordinates relevant notifications.

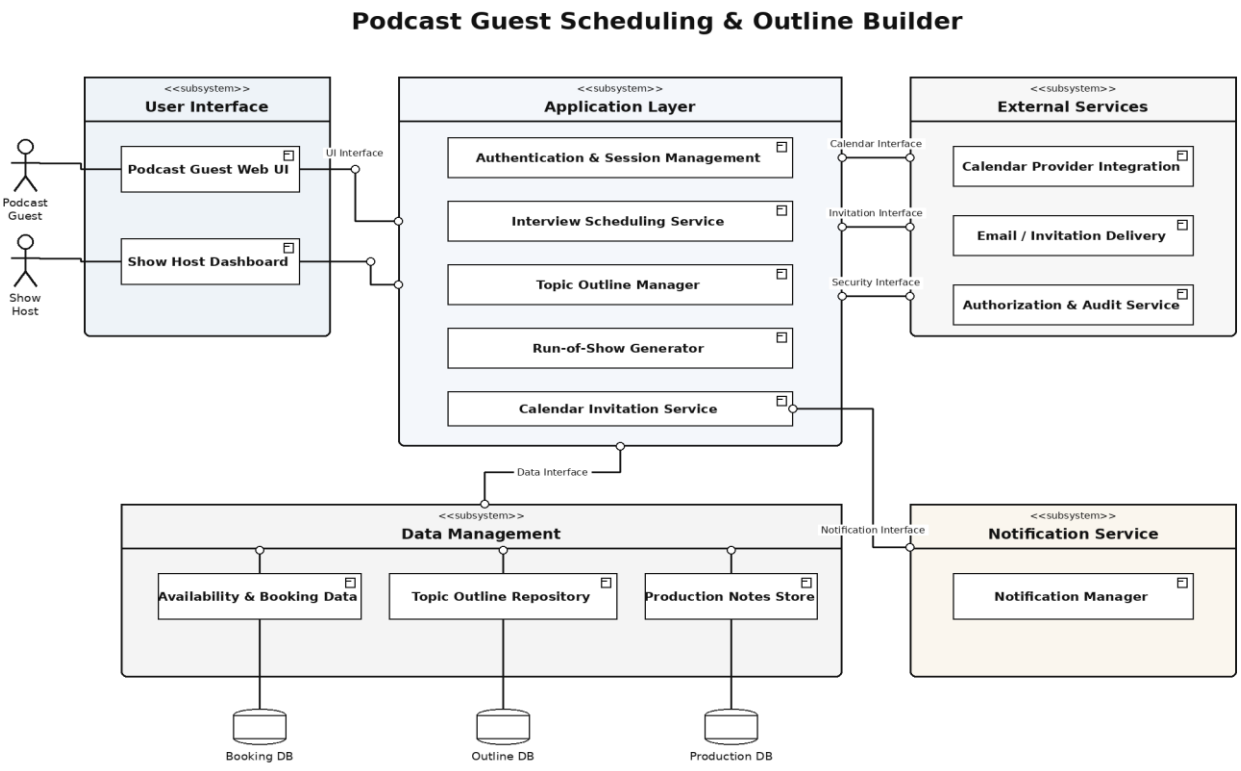


Figure 1. UML component diagram for Problem Statement #60

6. Component Responsibilities

Layer / Subsystem	Component	Primary Responsibility
Presentation	Podcast Guest Web UI	View slots, book interviews, and submit topics.
Presentation	Show Host Dashboard	Manage availability, bookings, and outlines.
Application	Authentication & Session Management	Authenticate users and manage sessions.
Application	Interview Scheduling Service	Check availability and confirm bookings.
Application	Topic Outline Manager	Manage and review guest outlines.
Application	Run-of-Show Generator	Generate timestamped production notes.
Application	Calendar Invitation Service	Send calendar invitations.
External Services	Calendar Provider Integration	Connect to external calendars.
External Services	Email / Invitation Delivery	Send booking confirmations and invitations.
External Services	Authorization & Audit Service	Check permissions and record actions.
Data Management	Availability & Booking Data	Store availability and booking details.
Data Management	Topic Outline Repository	Store guest and host-edited outlines.
Data Management	Production Notes Store	Store production notes.
Notification Service	Notification Manager	Manage booking and episode notifications.

Conclusion

Layered Architecture is suitable for the Podcast Guest Scheduling & Outline Builder because it separates the user interface, application rules, external integrations, notifications, and persistent data. It supports the required workflow of booking interviews, submitting and reviewing guest talking points, and generating timestamped production notes. This modular structure improves maintainability, supports security checks before data changes, and makes future expansion easier.